

034 实验指南

本实验从九参数网络的两次手算开始，再检查L2与解耦衰减，最后阅读一份完整、有限且可重跑的优化器比较。你需要交付能追踪计算和判断证据边界的记录，而不只是一个最低分。

1 环境与运行起点

先读正文第2至4节。实际核验环境为Linux CPU、Python3.12.14、PyTorch2.7.1+cpu、NumPy2.3.5、Matplotlib3.10.8，float64、单线程。版本是复现记录，不代表最新软件。以下Anaconda是学习者安装说明，没有声称各平台全新安装均已实测：

```
conda env create -f environment.yml
conda activate dl-unit-034
python experiment.py --output my-run
python -m unittest -v test_experiment
jupyter lab
```

在本讲目录打开experiment.ipynb，重启内核后从首格向下执行。Notebook会验证原始数据及原始结果，随后在临时目录重新计算并比较八份结果，不覆盖原始报告。若使用独立Python3.12 venv，先安装numpy==2.3.5、matplotlib==3.10.8、jupyterlab、ipykernel和nbformat，再从[PyTorch官方CPU版本渠道](#)安装torch2.7.1。无需GPU、模型下载、私人数据或付费API。

本讲代码不自动联网或安装软件。数据、参数配置和原始报告被SHA256固定。修改后固定脚本或绘图入口会明确拒绝，防止新参数配旧图。探索请另建文件，调用公开函数，并把新结果写入新目录；不要把数据摘要保护当成无法做实验。

2 纸笔串起两次更新

打开data/hand-example.json，只包含两行数据、九个初值和Adam参数。先不运行脚本，做以下工作。

1. 写出X、W、b、u、c的形状和参数总数；逐项计算四个Z、四个H、两个P、两个残差和两个半平方误差，再取平均。应得 $L_0=0.0221$ 。
2. 从L对逐行损失的1/2开始。计算dP、dH、ReLU掩码和dZ。画出W12的两条样本路径，确认一正一负并相加为.045；不是取绝对值相加。
3. 列出九个梯度。先算 $m_1=.2g_1$ 和 $v_1=.1g_1^2$ ，再修正，最后更新。对W11应得到.5197647058823529。
4. 用全部九个新值重算前向。第2次梯度必须来自新的H和新的输出权重，不能继续复用第1次梯度。
5. 使用 $m_2=.8m_1+.2g_2$ 、 $v_2=.9v_1+.1g_2^2$ ，分别除以.36与.19。再更新，第三次前向的平均半MSE应约.0000451447997794。

将你的表与my-run/hand-trace.json逐项比对。JSON的steps中，每步含forward_backward、adam和next_forward；forward_backward包含local、chain与contributions，不只给最终梯度。显示小数已舍入，数值核验以文件float64为准。

下面代码可直接在本目录的新Python文件或Notebook格运行。它重建同一条链，并打印每个参数变化：

```
import experiment as e
cfg, hand, data = e.load_inputs()
r = e.hand_trace(hand)
for s in r['steps']:
    a = s['adam']
    print('step', a['step'])
    print('loss', s['forward_backward']['loss'])
    for name, g, m, v, new in zip(
        r['parameter_order'],
        a['data_gradient'], a['m_hat'],
        a['v_hat'], a['theta_after']
    ):
        print(name, g, m, v, new)
    print('next loss', s['next_forward']['loss'])
```

本讲没有在ReLU零点做中心差分，手算轨迹所有仿射值都离零有正距离。若探索时跨过ReLU拐点，应先判断可微性，而不是立即判定autograd错误。

3 对epsilon和状态提出可检验问题

在Notebook中查看epsilon表和图4。比较 $g=1e-8$ 时epsilon取 $1e-8$ 与.001；再对照把同一epsilon写进根号内的结果。记录“只用于防0”的说法漏掉了什么。

mechanisms.json的delayed_gradient记录前999次零梯度、第1000次非零的例子。手算偏差修正后的m和v，核对更新大小为何可超过学习率。给定梯度序列只检查优化器，不要把它写成已经测量某网络的稀疏训练表现。

再看none_vs_zero：已有一次非零梯度之后，分别提供None与零Tensor。列出参数值、m、v和step的前后变化。问自己：如果一个条件分支暂时未用到，这一轮应当跳过它还是让它继续衰减？这是需要根据任务和实现决定的语义，不能只由“都没有梯度信号”来判定。

4 核查L2和解耦衰减

先用普通SGD把 $\theta - \alpha(g + \lambda \theta)$ 展开，说明为何它等于先收缩旧 θ 再走数据梯度一步。随后检查两个会失效的推广：带动量的耦合项进入历史；自适应的耦合项还进入平方状态。

阅读mechanisms.json的decay_comparison。初始(1,-2)，两次给定数据梯度(.1,2)和(-.2,1)，请至少逐项手算两个算法的第1步。然后解释第2步坐标1的修正动量为何异号。

pure_decay给出三条20步轨迹。只有数据梯度为0且初始状态为0，才可直接用theta0乘积公式描述整个更新。比较固定 $\alpha=.1$ 与 $.01$ 的最终参数，说明为何“解耦”不意味着“与学习率无关”。

5 审阅等预算搜索

data/config.json声明搜索空间、三个训练顺序seed、固定初值、衰减掩码及选择标准。data/samples.csv的split列固定48/16/16。先回答：

- 每个配置有多少实际optimizer step和样本呈现？96与768怎样从epoch及batch得来？
- 每家族9配置、3顺序，共多少次训练？为什么相同27次搜索仍不保证三个家族同样容易调好？
- 哪几个参数衰减？若把偏置也衰减，是否还是这份已报告实验？
- 如果把验证曲线最小点用于早停，会不会改变预先声明的选择规则？

完整81行search-grid.csv保留每个候选、顺序、训练/验证半MSE及样本顺序摘要。selection.json列出27个配置均值与3个最终选择。三个家族训练顺序按seed配对，可用摘要核对。测试数据从不传入fit和search接口；只有冻结选择后的evaluate_frozen才接收它。

各家族选中结果为SGD+动量 $\alpha=.03$ 、Adam和AdamW $\alpha=.01$ ，均选到衰减0。检查图7非零衰减列，解释“入选结果重合”与“算法一般相同”的区别。Adam的最佳值在学习率网格边界，这表示还不能声称搜索充分。不得为了获得AdamW获胜的图而事后换数据或只展示某个seed。

选择后共计算9个测试值，并额外重放9次已选训练以生成诊断曲线；这些重放不是新增候选，成本仍需披露。图8阴影只是三个训练顺序的范围，不是总体置信区间。没有计时数据，不得报告“同计算量更快”。

6 验证恢复与故障定位

state-check.json在固定手算批次上第2步截断，继续到第5步。完整state_dict恢复逐位相同；只恢复参数而清空状态，最大坐标差约.0428680。这里仅实际验证内存深拷贝，不涉及磁盘检查点、安全反序列化、随机采样或跨版本恢复。

15组测试会分别定位：前向/反向差异；Adam的状态和修正差异；L2耦合方式；预算和选择差异；输入与序列化保护。脚本先完成全部计算和序列化，后写结果，且使用每文件原子替换；它不保证整个输出集合在系统中途故障时一起成功或一起回滚。

7 应提交的研究记录

提交完整两次更新表、W12与u2各两条样本路径、epsilon位置错误的数值对照、L2与AdamW的差异说明、81行搜索预算与选优结果、冻结测试记录，并附运行日志和新结果目录。结尾写出至少三项不支持的主张，例如普遍优化器排名、真实数据泛化和统计显著性。先独立计算，再查answers.pdf核对。